טכנו״צ DS | 77926

דו״ח תרגיל 2 – סיכום פרק ML

מגישים: אריאל מנלה (216441915)

וטל רמות (329514061)

תאריך: 3.5.26

Part A – Data Representation & Preprocessing

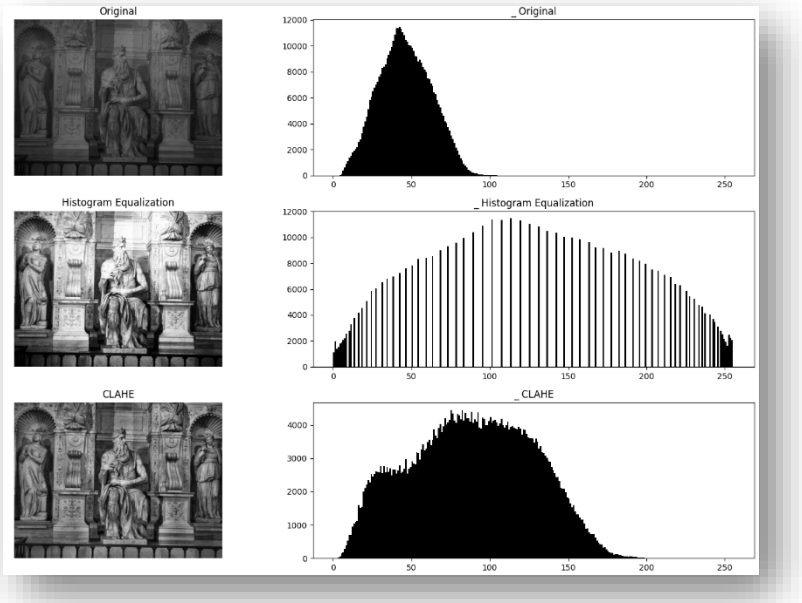
בהסתכלות על התמונות, ניתן להבחין שהן כולם תמונות של אנשים, על רקע דומה, כולן בשחור לבן ומכילות ברובן חלק מסוים של פרצוף האדם הרלוונטי. לכן, מיותר לחפש פיצ׳רים המבדילים בין רקעים למשל, או בין המיקומים של האנשים בתמונה (שכן אין בהם תלות בזהות האדם שבתמונה). אז מה בכל זאת נרצה לעשות במקום ללכת brute-force עם הדאטה ולשפוך למודלים?

נרצה לבצע עיבוד מקדים הבנוי משני חלקים עיקריים – הראשון הוא לנרמל את הגוונים שבתמונה על מנת להגדיל את הניגודיות ולהקל על המודלים שנאמן עליהם את הדאטה לזהות בו תבניות ייחודיות לפרצופים שונים, והצעד השני הוא לחלץ מהתמונה פיצ׳רים שעושים שימוש בגיאומטריה של התמונה (למשל ע״י חישוב גרדיינטים בדידים באזורים שונים בתמונה) על מנת לסייע למודלים לעבוד עם כמות נמוכה יותר של פיצ׳רים פר דגימה, שנותנים אינדיקציה טובה יותר על הפרצוף המופיע בכל תמונה (יותר מאשר תת קבוצה שרירותית של פיקסלים...).

כעת נציג את שתי הפעולות שתיארנו לעיבוד המקדים של המידע:

1. **CLAHE** (Contrast Limited Adaptive Histogram Equalization)

זוהי בעצם שיטה לחידוד הניגודים בין הצבעים בתמונה, כפי שנראה בתמונה הבאה:



האלגוריתם מבצע חלוקה של התמונה לבלוקים, ולכל אחד מהם עושה היסטוגרמה של עוצמת פיקסלים מחושבת עבור כל בלוק, המייצגת את התפלגות רמות הבהירות בתוך אותו אזור ספציפי. הוא ממרכז אותה, ואז כדי למנוע מעברים פתאומיים בין בלוקים בעת שחזור התמונה, מופעל אינטרפולציה דו-לינארית. זה מחליק את הבדלי הניגודיות בין אזורים.
לבסוף, כל הבלוקים מאוחדים לתמונה הסופית.
בעזרת שלב זה, יהיה קל יותר לאלגוריתמים להפריד בין הצורות השונות של הפנים, ולראות הבדלים בין החלקים השונים בתמונה.

בנוסף מתבצע נירמול באופן הבא:

$$X_{normalized} = \frac{X-\mu}{\sigma}$$

כלומר, "למרכז" את הווקטורים של התמונה לפי סטית תקן וממוצע, כך שיהיה קל להפעיל עליהם אלגוריתמים של עיבוד מידע ולהשוות בינהם.

2. **HOG** (histogram of oriented gradients)
האלגוריתם מביט עבור כל פיקסל על השכנים שלו, ומחשב את הגרדיינט שלו בכיוון x ו-y.

$$g = \sqrt{g_x^2 + g_y^2}$$
$$\theta = \arctan \frac{g_y}{g_x}$$

כמו ה-**CLAHE**, הוא גם מחלק את התמונה לאיזורים קטנים מאוד, ולכל אחד נותן ווקטור שמצביע על כיוון השיפוע העיקרי. כך, הוא מזהה מתי יש פינות חדות, ויכול להבדיל בין אובייקטים.

לבסוף, אנו נותרים עם מערך מספרים עבור כל תמונה. ה-**CLAHE** ווידא שהערכים לא תלויים בתאורה, ה-**HOG** נתן מידע על איפה יש קווים חדים וצורות, והנרמול ווידא שיהיה אפשר להשוות בינהם בשלב הבא.

3. <u>[הערה חשובה] דחיסת המידע לשיפור זמן הריצה:</u>

כשניסינו להתחיל להריץ בדיקות, נתקלנו בזמני ריצה אדירים בשל כמויות מידע עצומות, כי אלגוריתם ה-**HOG** המחושב בצורה נאיבית, מחלץ וקטור ארוך מאוד עבור כל תמונה (מעל 1,000 מימדים). לכן, ביצענו אלגוריתם **PCA** על מטריצת הדגימות.

ה-**PCA** לוקח עבור כל תמונה את אלף (בערך) הפיצ'רים שהוציא ה-**HOG** בתמונה. ומוציא עבורם סט של 50 פיצ'רים המכסים את מירב השונות של הדאטה על פני תת המרחב $\mathbb{R}^{50}$ (זו המשמעות של ביצוע אלגוריתם זה). בנוסף, צעד זה מסייע למתן "רעש" שמופיע בוקטור ה-HOG (ככל שיש פחות דאטה יש פחות התייחסות לרעש) וכן עוזר למנוע Overfitting כיוון שהדאטה פשוט יותר ואינו ייחודי עבור כל תמונה (לפחות לא כמו שימוש נאיבי במערך ערכי הפיקסלים שלה).

Part B – Supervised Models: Introduction & Motivation

המודלים שבחרנו לעשות בהם שימוש בתרגיל הם:

1. עץ החלטה.
2. רגרסיה ליניארית (softmax).
3. Generalized SVM (ספויילר: זהו המודל המוצלח ביותר בפער).

רגרסיה ליניארית

נמיין את הדאתה ל-28 מחלקות, באמצעות שימוש בפונקציית softmax שלמדנו עליה בהרצאה הראשונה בקורס. $P(y|x,\theta_i)$ הוא הסתברות דיסקרטית לאחד מתוך 28 ערכים, ולכן נרצה 28 ווקטורים θ_i, כאשר כל אחד ייצג את הפרמטרים המתאימים ל-$P(y=i\ |x,\theta)$. עבור נרמול (כאשר הם ייסכמו כולם ל-1 וכמו כן ייצגו את גודל ההסתברות נכון), יש להשתמש בפונקצית ה-softmax. לבסוף, המודל "לומד" על ידי עדכון המשקלים בעזרת אלגוריתם **Gradient Descent**, וכך מגדיל את ההסתברות של מיון האדם הנכון בתמונה.

1. **היפר – פרמטרים:**

- <u>קצב לימוד</u>: קצב הלמידה קובע את גודל הצעד בכל עדכון של הפרמטרים. אם גודל הצעד גדול מדי, אנו עלולים לפספס את נקודת המינימום (ההתאמה האופטימלית), ואם הוא נמוך מדי, ייקח לאלגוריתם המון זמן לרוץ, והוא עלול גם להגיע לנקודת מינימום מקומית זניחה במקום להיפותזה האופטימלית באמת.
- <u>מספר איטרציות</u>: כמות הפעמים שהאלגוריתם עובר על כל הנתונים. כיוונו זאת באמצעות "עצירה מוקדמת": הפסקנו את האימון ברגע שדיוק הנתונים גבוה מספיק.

2. **רגולריזציה:**

- ב-Softmax Regression, אנחנו מוסיפים "קנס" (Penalty) למשקלים של המודל. אם משקל מסוים הופך להיות גדול מדי (נגיד, המודל החליט שפיקסל בודד בקצה התמונה הוא ההוכחה היחידה שזה "אדם 1", או כניסה אחת של הוקטור HOG במקרה שלנו), הרגולריזציה "קונסת" אותו ומכריחה אותו להשתנות. התוצאה: המודל נאלץ להסתכל על הרבה תכונות קטנות במקום על תכונה אחת גדולה, מה שמונע ממנו לעושת overfitting – כלומר, אנו מעודדים מודלים פשוטים יותר. לשיטה זו שעושה שימוש ישיר בגודל של וקטור המקדמים של המודל לקביעת המורכבות שלו, קוראים L^2 ונתייחס אליה עוד בהמשך.

3. **Bias and variance – הטיה ושונות**

בד"כ יש למודל רגרסיה לינארית ביאס גדול במקרה של דאטה גדול ולא-לינארי שנעשה בו שימוש בתרגיל זה, ובהתאם ל-bias\variance tradeoff, השונות שלו קטנה (פירוט בחלק F).

4. **למה המודל יכול להיות טוב לביצוע המשימה:**

- אנו מממיינים בעיקר לפי HOG, כלומר לפי כיווני גרדיאנט. למשל, למעגל יש כיווני גרדיאנט מסוימים, לריבוע אחרים... כך שסך הכל, אנו מחפשים תבניות. מודל ליניארי פשוט מחפש "צירוף משקלים" שמתאים לתבניות האלה. הוא מחשב סכום משוקלל של כל אזור בתמונה. אם

האזורים שמאפיינים "עיגול" מקבלים משקל גבוה, המודל יזהה את הכדור בקלות. זו גישה ישירה ומהירה מאוד לזיהוי צורות גיאומטריות פשוטות.

- יעיל מבחינה חישובית – כפל מטריצות הוא הרבה יותר יעיל מבניית עצי החלטה ברקורסיה למשל, או ממודלים מורכבים אחרים. יש לנו הרבה תמונות ופיקסלים, ולכן פשטות ויעילות הינה הכרחית.
- קל לעשות לו רגולריזציה – בעזרת שיטת ה-L^2 שהצגנו, ניתן לדאוג שהמודל לא יעשה overfitting.

עץ החלטה

עץ החלטה הוא מודל חיזוי שממפה תכונות באמצעות סדרה של פיצולים בינאריים. אלגוריתם CART שלמדנו בתרגול 2 מאפשר בנייה של עץ החלטה בינארי באמצעות תהליך הדרגתי, מלמעלה־למטה, המבצע חלוקות רקורסיביות של מרחב הדגימות כך שבכל צומת נבחר הפיצול שמקטין בצורה המרבית את ה-impurity (נקבעת לפי אנטרופיה או מדד ג'יני, כפי שראינו בתרגול): בכל צומת, CART בוחן את ה-impurity עבור כל תכונה j ועבור כל סף פיצול אפשרי th, ובוחר את הפיצול $(j\ ,t)$ שמביא להפחתת ה־Impurity הגדולה ביותר.

1. **היפר-פרמטרים:**
   - עומק מקסימלי: קובע כמה רמות יהיו לעץ.
   - מינימום דגימות לפיצול: המינימום הנדרש של דגימות בצומת כדי לבצע פיצול נוסף.

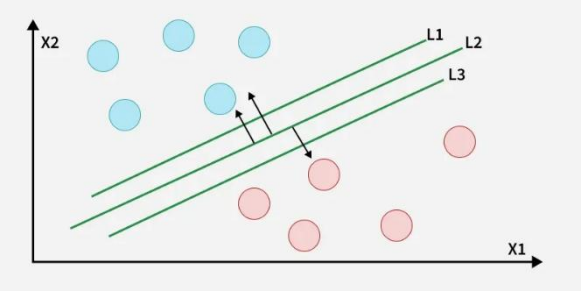
2. **רגולריזציה:** בקרת המורכבות באמצעות הגבלת עומק העץ לפני שהוא הופך למורכב מדי. מנגנון רגולריזציה שמיושם לאחר שהעץ נבנה בצורה מלאה.
   יכלנו להשתמש בגיזום: במקום לעצור את גדילת העץ מראש, CART מגדיל אותו עד למקסימום המותר, ואז גוזם חלקים שנראים כפחות מועילים על בסיס מדדים שהוגדרו מאש. עם זאת, זהו תהליך ארוך יותר, ויש לנו המון נתונים, כך שבחרנו לקצץ את גודל העץ מראש וכך למנוע overfitting.
   בנוסף, בחירת מדד ה-Entropy/Information Gain מבטיחה שהפיצולים שנבחרים הם המשמעותיים ביותר סטטיסטית.

3. **Bias and variance**
   ככלל למודל זה (ובמיוחד כאשר מספר הרמות בו הוא גבוה) יש ביאס קטן ושונות גדולה, כיוון שבמהות שלו הוא מבצע הפרדה קפדנית בכל שלב באלגוריתם CART בהתאם לעיקרון של הקטנת ה-impurity – תהליך שעלול להיות שונה מהותית עבור שינויים קטנים בדאטה ולהוביל לשונות גבוהה על אף ה-bias הנמוך (למעשה זה בדיוק ה-bias\variance tradeoff במיטבו).

4. **למה הוא מתאים לפרוייקט?**
   בעיקר כי הוא מדויק מאוד – הוא בודק את כל האופציות, ובעזרת בדיקה אם משהו הוא מעל או מתחת לסף, מקבל החלטה כל פעם. לכן, הוא מתאים לסיווג לקטגוריות שונות.

SVM

שיטה זו מנסה להפריד בין 2 קבוצות של נתונים, כמו בתמונה:



מה קורה כשהנתונים לא ניתנים להפרדה בקו ישר? למשל, אם הנקודות הכחולות נמצאות בעיגול פנימי והאדומות מקיפות אותן? כאן נכנסת שיטת המשתמשת ב-kernel: המודל "מרים" את הנתונים לממד גבוה יותר (למשל, הופך דו-ממד לתלת-ממד). במימד הגבוה הזה, ניתן למצוא מישור שיפריד ביניהם בקלות.

1. היפר-פרמטרים:

C (penalty): קובע את ה-Tradeoff בין הגדלת המרווח (Margin) לבין מזעור שגיאות הסיווג. כלומר, קובע כמה ה-SVM רך או קשה, כמה הוא מוכן לקבל שגיאות (כמה הוא מעדיף קו ישר שלא מפריד מושלם בין הקבוצות, לעומת קו מפותל וספציפי שמפריד ממש טוב בין הנתונים).

Gamma: קובע את טווח ההשפעה של דגימה בודדת. ערך גבוה אומר שרק דגימות קרובות משפיעות. רלוונטי רק לSVM בו אנו משתמשים בkernel, כלומר כאשר "עולים" למימד גבוה יותר.

2. רגולריזציה ובקרת מורכבות:

הרגולריזציה ב-SVM היא מובנית דרך ה-**Margin Maximization**. **ה**מודל מנסה למצוא את העל-מישור עם ה"מרחק" הגדול ביותר מהנקודות. בנוסף, הפרמטר C משמש כבקר רגולריזציה – C קטן מייצג רגולריזציה חזקה – הוא מכריח את המודל לקבל שגיאות ולא להיות סופר ספציפי ולהפריד באופן מושלם בין נתונים.

3. התנהגות Bias/Variance צפויה:

בשימוש עם RBF Kernel, למודל יש **Bias** נמוך כי הוא יכול ללמוד גבולות הפרדה מורכבים מאוד במרחב גבוה. בזכות הפרמטר C, ה-variance שלו נשלט ולא גבוה מדי.

4. **למה הוא מתאים לפרויקט?**

זהו מודל יותר מורכב מעץ ההחלטה ומהרגרסיה הליניארית, ולכן מאפשר "צורות נוספות" להפרדה, בניגוד למשל למודל הרגרסיה הליניארית. בנוסף, האו יודע לאזן טוב בין bias ל-variance ולכן ימנע מ overfitting וגם מ-underfitting באפקטיביות. בנוסף, הוא מתוכנן מראש לעבוד במימדים מאוד גבוהים, שאיתם אנחנו עובדים בפרוייקט בשל עיבוד נתונים בעזרת HOG (לאחר ה-PCA, וקטורי הפיצ'רים עבור כל דגימה המסמלת תמונה הם בעלי 50 כניסות).

Part C – Regularization & Model Complexity

רגרסיה ליניארית

מידת המורכבות של המודל נובעת מהמשקולות- הרי הנוסחה עבור אדם בודד נראית כך:

$$Z = X_0 + W_1X_1 + \cdots + W_nX_n$$

משקולת גבוהה אומרת שהפיצ'ר המסוים הזה מאוד אופייני לאדם. ככל שהמשקולות גדולות וספציפיות יותר, המודל הופך לספציפי יותר – מנסה להתאים קו שעובר בדיוק דרך הנתונים. מודל פשוט הוא מודל שבו המשקולות קטנות ומפוזרות באופן שווה. אם נהיה קיצוניים מדי לאחד הצדדים, נקבל מודל מדויק מדי או לא מדויק מספיק - overfitting או underfitting.

כדי לשלוט בזה, יש 2 דרכים עיקריות. הראשונה היא אחד מההיפר פרמטרים של רגרסיה ליניארית: קצב הלימוד, שקובע את גודל הצעד בכל עדכון של הפרמטרים. אם גודל הצעד גדול מדי, אנו עלולים לפספס את נקודת המינימום (ההתאמה האופטימלית), ואם הוא נמוך מדי האלגוריתם עלול להגיע לנקודת מינימום מקומית זניחה במקום להיפותזה האופטימלית באמת.

הדרך השניה היא באמצעות ביצוע רגולריזציה על המשקולות שאליהם המודל מבצע התאמה, ע"י הוספת איבר הפסד מהצורה של מטריקת L^2:

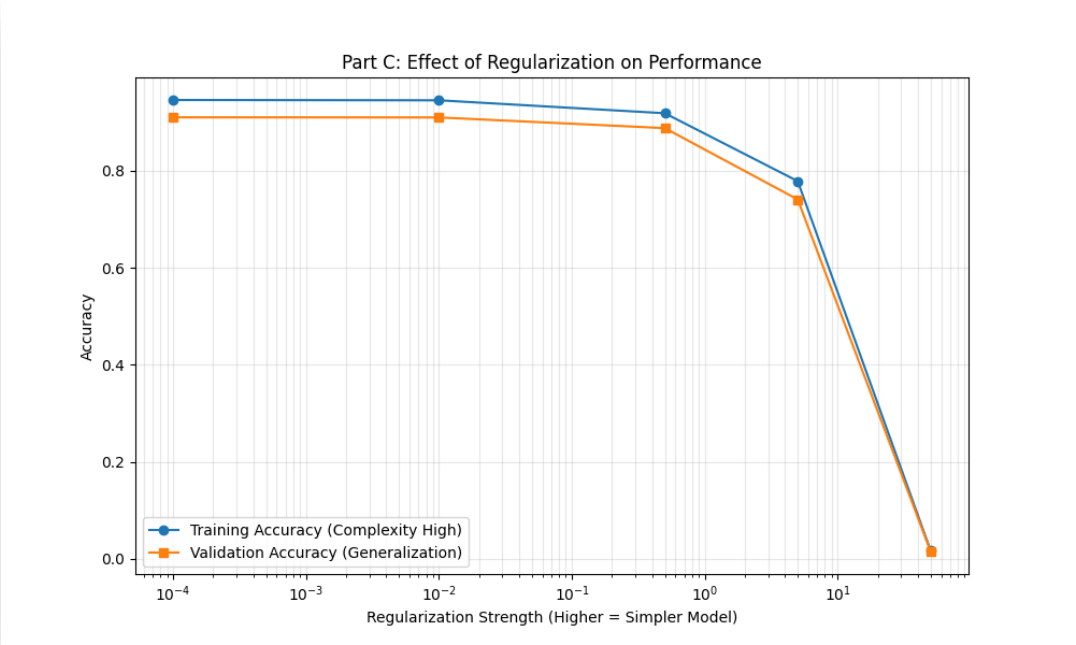
Loss with L2 Regularization

$$\mathcal{L}(\theta) = \frac{1}{N}\sum_{i=1}^{N} Cost(y_i, f_\theta(x_i)) + \lambda\|\theta\|_2^2$$

היא מוסיפה עוד חלק לפונקצית הloss. כלומר, כאשר אנו מוסיפים משקלים גדולים, פונקצית ההפסד תגדל, וכך למודל יש מוטיבציה לשמור על המשקלים שלו לא קיצוניים מדי, אלא יחסית אחידים.

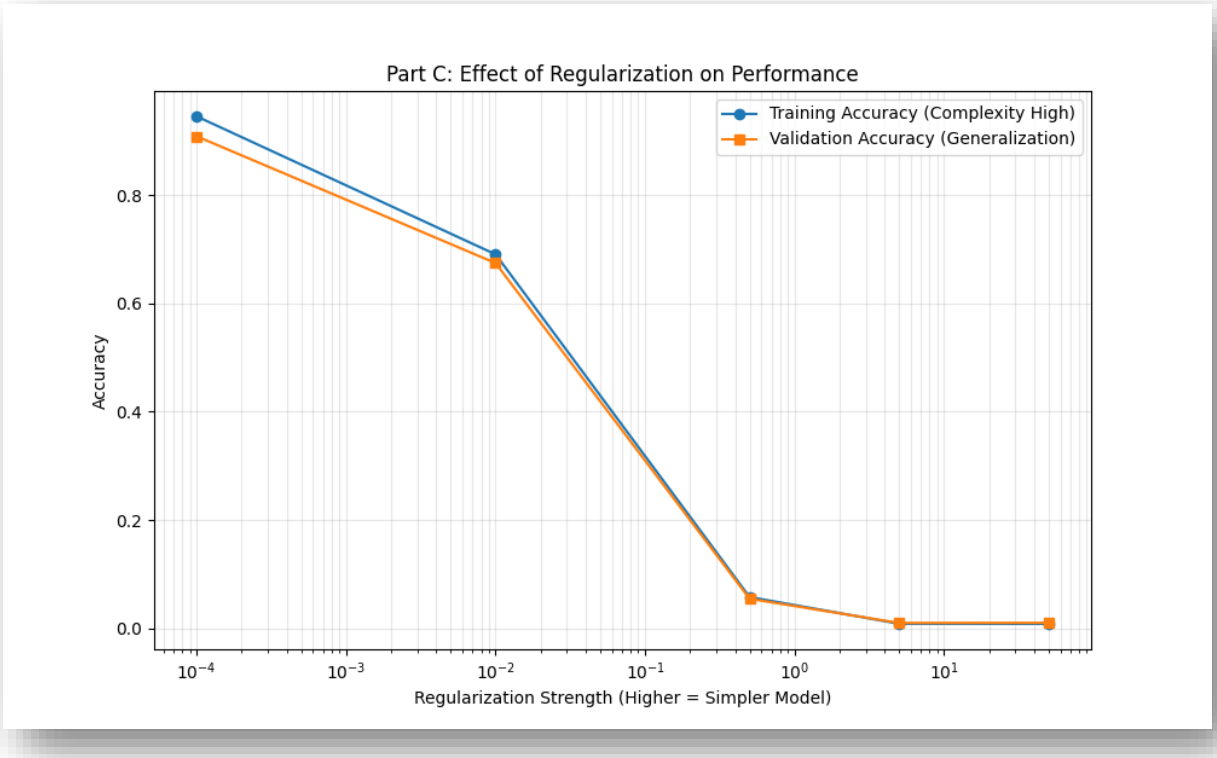
שיטה זו היא מאוד יעילה. L_1 $(lasso)$ לדוגמא היא שיטה דומה, רק שנוטה לאפס משקלים לגמרי ולכן הינה פחות אפקטיבית.

כדי לנסות להבין איך L_2 משפיע: ב-L_2 הוספנו לגרדיינט את הערך self.weight * self.reg, אז נריץ את המודל לדוגמא (נחלק את המודל שלנו לקבוצת למידה וקבוצת בוחן על פי הנתונים המסווגים) על כמה ערכים של reg, כלומר ניתן penalty שונה כל פעם, ונראה איך זה משפיע על דיוק המודל.



הקו הכחול הינו דיוק לגבי תמונות שהוא כבר ראה, והכתום על תמונות שלעולם לא ראה. ניתן לראות שכשערך הרגולציה קטן, יש דיוק די גבוה, ואילו כשהוא גבוה מדי, אפשר לראות שהדיוק קטן לגמרי, כי המודל כבר לא מסוגל להיות מורכב מספיק כדי להפריד בין האנשים מספיק טוב.

כדי להמחיש למה לא בחרנו ב-L_1, נריץ את אותו הקוד גם כשהוא הרגולציה על פונקצית ה-loss:



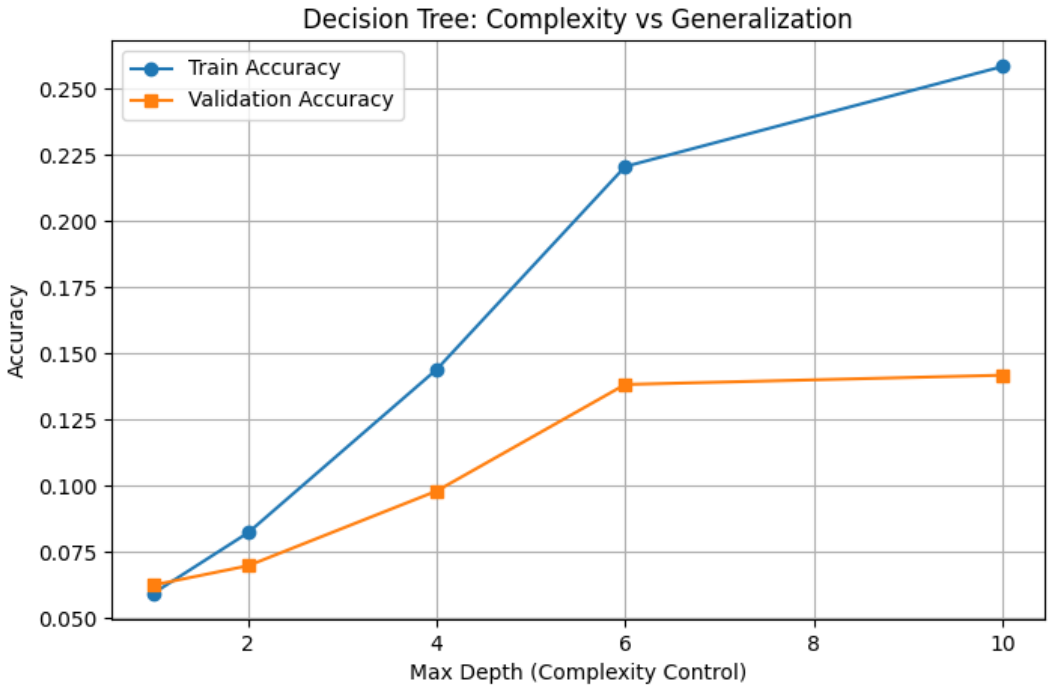
ניתן לראות שמהר מאוד המודל אינו מורכב מספיק, ושהדיוק נמוך אפילו עבור תוצאות שהוא כבר ראה.

עץ החלטה

מה שקובע את המורכבות שלו הינו מבנה העץ - כמה הוא גדול, וכמה פיצולים יש בו. אם העץ מאוד גבוה, הוא יודע "את כל המקרים", ואז ייתכן שהוא מסווג את אותו אדם לשני אנשים שונים- עושה overfitting על הנתונים בעצם. אם העץ נמוך מדי, זה אומר שהוא לא יודע לסווג מספיק טוב את סוגי האנשים, כי אין בו מספיק פיצולים, "שאלות", כדי להבדיל בין כל הקטגוריות.

המנגנונים לרגולציה:

הראשון הוא גודל העץ, כפי שהוסבר. הוא מוגדר מראש, כך שהעץ מפסיק להתפצל אם הגיע לגובה מסוים. ננסה להריץ את המודל כמו מקודם, על גבהים שונים:



ניתן לראות שכשיש רק פיצול 1, המודל מאוד לא מדויק, וטועה גם על הדתא שהוא כבר למד. לאחר מכן, הדיוק הולך ועולה ככל שהפיצולים ממשיכים. נשים לב שהדיוק של הדוגמאות שהמודל עוד לא ראה, הקו הכתום, נשאר יציב בעוד הדיוק על הדוגמאות שהמודל כבר מכיר, הכחול, ממשיך לעלות. זה אומר שאם נגביר את הפיצולים נגיע ל-overfit, ולכן בערך 6-8 פיצולים זה כנראה המספר האופטימלי.

רגולריזציה שיכלנו לעשות במקום הינה לבנות את כל העץ ואז לקצר אותו בסוף לפי מה שנראה פחות רלוונטי-זה דורש יותר זמן ריצה ויותר מסובך לקוד, ומכיוון שהם עושים דברים דומים, בחרנו להגביל את אורך העץ מאש.

רגולריזציה נוספת היא מינימום דוגמאות הנדרשות לפיצול – כשהוא קטן, העץ ימשיך להתפצל עד שכל תמונה בודדת תקבל ענף משלה, שזהו overfitting בשיא תפארתו. במצב ההפוך, העץ לא מקבל החלטות על סמך קבוצות קטנות מדי של תמונות, כי אולי זה מקרי, וכך יש הכללה מספקת. הכללה גדולה מדי לא תאפשר מודל מורכב מספיק.

SVM

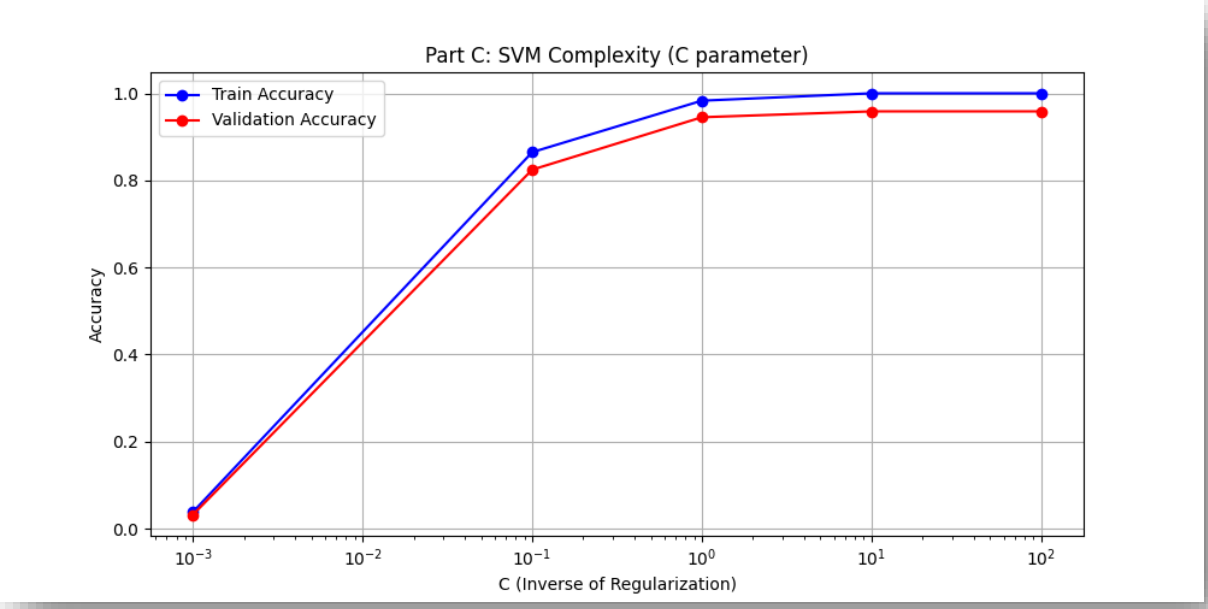
מודל חזק בהרבה מהליניארי כי הוא יודע לייצר גבולות הפרדה עקמומיים ומורכבים.

הפרמטרים ששולטים בו הינם C ו-γ, עליהם דנו בחלק הקודם וכעת נרחיב את הדיון.:

C קובע את ה-Tradeoff בין הגדלת המרווח (Margin) לבין מזעור שגיאות הסיווג. כלומר, קובע כמה ה-SVM רך או קשה, כמה הוא מוכן לקבל שגיאות (כמה הוא מעדיף קו ישר שלא מפריד מושלם בין הקבוצות, לעומת קו מפותל וספציפי שמפריד ממש טוב בין הנתונים). אם הוא יותר קשה, כלומר לא מוכן לקבל שגיאות, זה יכול להוביל ל-overfitting וקו ספציפי מדי. SVM רך יעשה את האפקט ההפוך.

γ קובע את טווח ההשפעה של דגימה בודדת. ערך גבוה אומר שרק דגימות קרובות משפיעות. רלוונטי רק ל-SVM בו אנו משתמשים ב-kernel, כלומר כאשר "עולים" למימד גבוה יותר. גאמא גבוה מדי אומר שרק דגימות קרובות משפיעות זו על זו, ואז המודל מאוד מורכב. גאמא נמוך אומר שכל הדגימות משפיעות על כולן, ואז המודל יוצא מאוד פשוט.

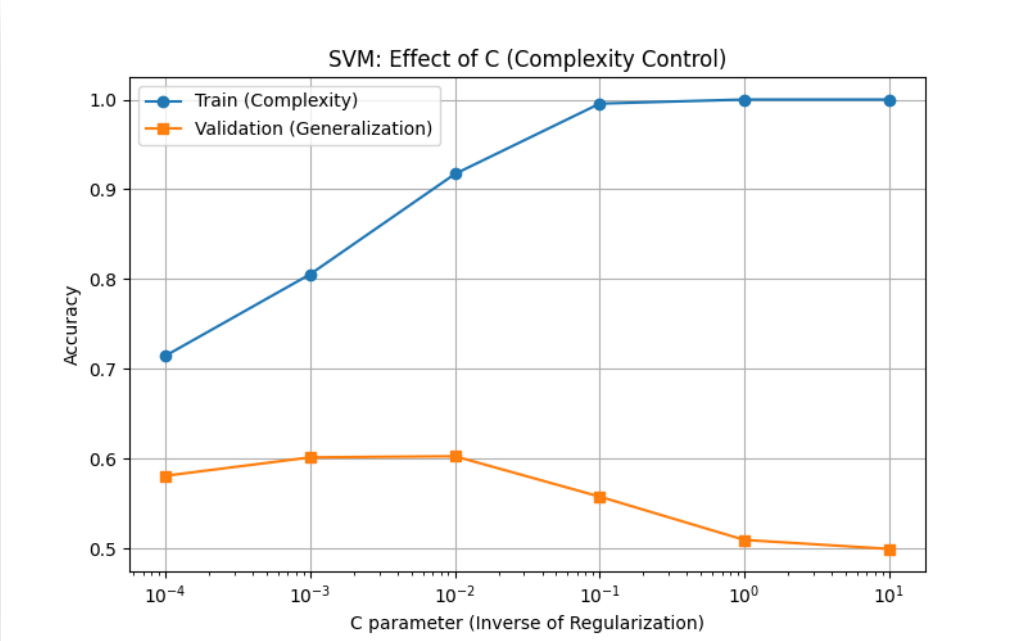
נביט בגרף שבו מופיע הדיוק של המודל SVM על קבוצות ה-Train וה-Val, כש-C הוא הפרמטר ששינינו:



אכן, C תרם, וככל שהוא גדל, גם הדיוק גדל. כרגע, נראה שהוא המודל עם אחוזי הדיוק הכי גבוהים, והוא עוד עולה.

לשם הפשטות, חשבנו לנסות מודל SVM ליניארי, כלומר רק בשני מימדים וללא ה-kernel, אך כפי שניתן לראות בגרף הבא, זה היה מודל הרבה פחות מדויק, כצפוי. כשהוא נותן דיוק גבוה לערכים שהוא "מכיר",

מתקבל דיוק נמוך לערכים שהוא רואה לראשונה, וכך בעצם אנו מבינים שהוא עשה overfit , ולא יודע להתאים היפותזה טובה באופן אמין.



Part D – Model Selection (with Nested K-Fold Cross-Validation)

בחלק זה השתמשנו בוריאציה משוכללת יותר של מה שנלמד בהרצאה השנייה של קויצקי (בחלק שעסק בבעיית הכללה של מודל למידה) – והיא nested K-fold Cross Validation.

זוהי שיטה לשיפור ההכללה (Generalization) של המודל שלנו, כלומר לוודא שהוא מגיע ל-accuracy גבוה גם בקבוצת ה-train אבל גם בקבוצות ה-validation וה-test, שהרי בסוף מטרתנו היא שהמודל יגיע לדיוק גבוה דווקא בדאטה שהוא לא מכיר.

במסגרת שיטה זו מבצעים פרקטיקה בעיקרון דומה ל-Hold-Out Cross Validation, כאשר בוחרים את ההיפר-פרמטרים האופטימליים עבור כל מחלקת היפותזות בהתאם לביצוע שלהם על קבוצת ה-validation שהיא בהגדרה זרה לקבוצת ה-train עבור ההיפותזה הספציפית שנבדקה.

כדי לתאר את שיטת nested K-fold CV שמתוארת בהוראות התרגיל, נציג כאן את הפסאודו-קוד המתאים לה, שהוא הרחבה של שקף 71 מההרצאה השנייה של קויצקי.

Algorithm: Nested K-Fold Cross Validation

Input: Set of model classes $M = \{M_1, \dots, M_d\}$, dataset D of size m, number of outer folds K_{out}, number of inner folds K_{in}.

1. Split D into K_{out} disjoint subsets $T_1, \dots, T_{K_{out}}$ of size $\frac{m}{K_{out}}$ (called "outer folds")
2. For l=1,…,K_{out}:
   a. Define the set $S_l = D \setminus T_l$ of size $m \cdot \frac{K_{out}-1}{K_{out}}$
   b. Split S_l into K_{in} disjoint subsets $S_1, \dots, S_{K_{in}}$ of size $m \cdot \frac{K_{out}-1}{K_{out}K_{in}}$
   c. For $i = 1, \dots, d$: (iterating over models M_i)
      i. For j=1,…,K_{in}:
         1. Train M_i on $S \setminus S_j$ to get hypothesis h_{ij}
         2. Test h_{ij} on S_j to get ε_{ij}
      ii. Estimate generalization error $\varepsilon_i = \frac{1}{K_{in}} \sum_{j=1}^{K_{in}} \hat{\varepsilon}_{ij}(S \subset S_j)$
   d. Select $H_l \triangleq M_i$ where $i = argmin_{i=1}^{d}(\varepsilon_i)$
   e. Retrain H_l on the entire set S_l
   f. Test H_l on the outer fold T_l and receive $\varepsilon_l = \hat{\varepsilon}_l(T_l)$.

3. Return $H \triangleq H_l$ where $l = argmin_{l'=1}^{K_{out}}(\varepsilon_l)$

נשים לב שלכל $l \in \{1, \dots, K_{out}\}$ השורות $2a - 2f$ הן בדיוק המימוש של K-fold CV, כאשר מה שהוספנו במקרה של nested K-fold CV הוא בדיקה סופית של המודל לאחר הבחירה שלו, באמצעות דאטה שהוא לא ראה מעולם – בניגוד לגירסה הפשוטה יותר (לא nested) שבה כל אחת מהקבוצות S_j ששומשו לוואלידציה היא למעשה משומשת לאימון של כל שאר המודלים.

זה ההבדל בין השיטות – השיטה הפשוטה יותר מאפשרת **בחירה** אופטימלית של המודל הטוב ביותר מתוך קבוצה נתונה של מודלים, ואילו השיטה המורכבת יותר (בה השתמשנו כאן עפ״י הוראות התרגיל) מאפשרת **הערכה** בצורה טובה של איכות המודל הנבחר.

<u>כיצד הבחירה של K משפיעה על תהליך הואלידציה?</u>

בשיטת ה-nested K-Fold CV יש שימוש בשני קבועים שונים עבור K בלולאה החיצונית (K_{out}) ובלולאה הפנימית (K_{in}) כאשר כל אחד מהם קובע את גודל החלוקה לתתי קבוצות זרות ושוות בגודלן (K_{out} קובע את החלוקה של הדאטה כולו ל-outer folds שמשמשים ל-testing של המודל בשלב 3 באלגוריתם, ואילו K_{in} קובע את החלוקה של הדאטה ללא ה-outer fold לדאטה לצורך אימון ולצורך ואלידציה של המודלים השונים).

ככלל, גודלו של K באלגוריתם K-Fold מייצר trade-off בין כמות האבליואציות שמבצעים על כל מודל M_i (בלולאה 2c באלגוריתם) לבין כמות הדאטה בכל אבליואציה כזו (שבה מחשבים את ε_{ij} לכל $j = 1, \dots, K_{in}$).

במקרה שלנו ה-tradeoff שתואר מתאים ל-K_{in} ואילו גם K_{out} מייצר trade-off – בין כמות ה- outer folds שמשמשים לבדיקה לבין הגודל של כל אחד מהם. הבחירה האופטימלית של K_{in}, K_{out} תלויה כמובן בדאטהסט – בעיקר בגודלו ובכמות הדאטה המינימלית שמאפשרת לאמן מודל מסוים.

<u>תוצאות ההשוואה</u>

עפ״י תוצאות ההשוואה, המודל הטוב ביותר (ללא עוררין) הוא ה-SVM שייבאנו בתרגיל מספריית sklearn עם הפרמטרים $C = 100, \gamma = "scale"$. בחלק הקודם הסברנו ש – γ אחראי על מידת ההשפעה של דגימות אחת מהשנייה (ככל ש – γ גבוה יותר כך מידת ההשפעה של דגימות רחוקות היא נמוכה יותר). במצב $scale$ הערך של γ נקבע ע״י:

$$\gamma = \frac{1}{n_{features} \cdot Var(X)}$$

כאשר $Var(X) = \frac{1}{md}\sum_{i=1}^{m}\sum_{j=1}^{d}\left(x_{ij} - \mu_{global}\right)^2$ הוא השונות של כל איברי המטריצה $X \in \mathbb{R}^{m \times d}$ (שורותיה הן הדגימות, כל דגימה היא וקטור פיצ׳רים של אחת התמונות).

בקוד שלנו חישבנו בדיוק את הערך הזה וקיבלנו $\gamma_{scale} = 0.00114$. בהשוואה ביצענו הרצה על כל 20 הקומבינציות של המודלים עם פרמטרים בקבוצה:

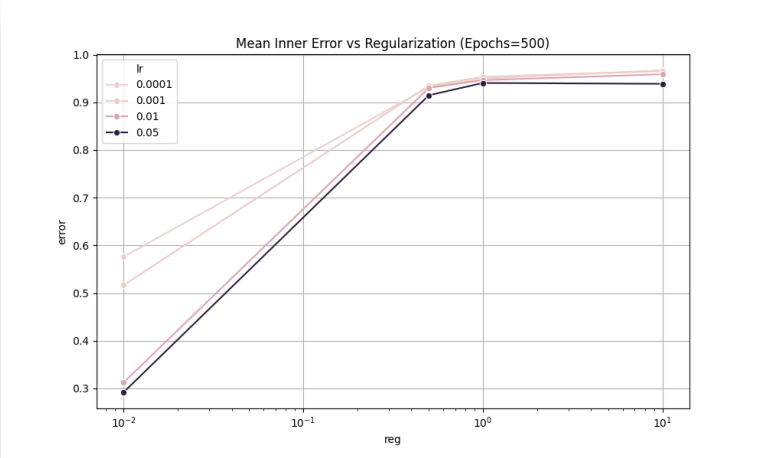
$$(C, \gamma) \in \{10^0, 10^1, 10^2, 10^3, 10^4\} \times \left\{10^{-4}, \underbrace{10^{-3}}_{scale}, 10^{-2}, 10^0, 10^2\right\}$$

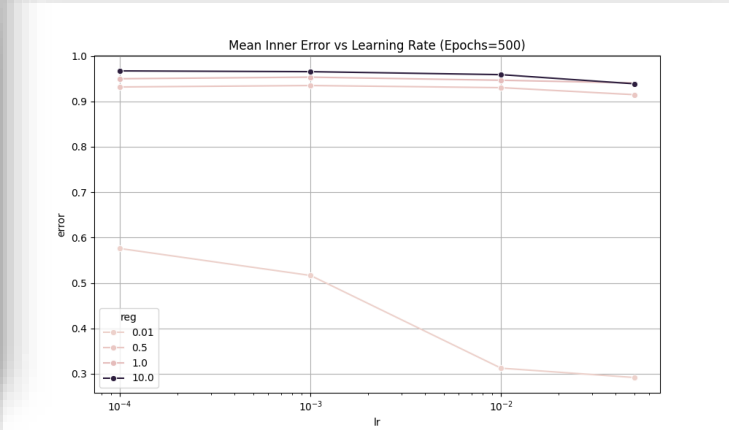
<u>השפעת ההיפר-פרמטרים על דיוק המודלים השונים</u>

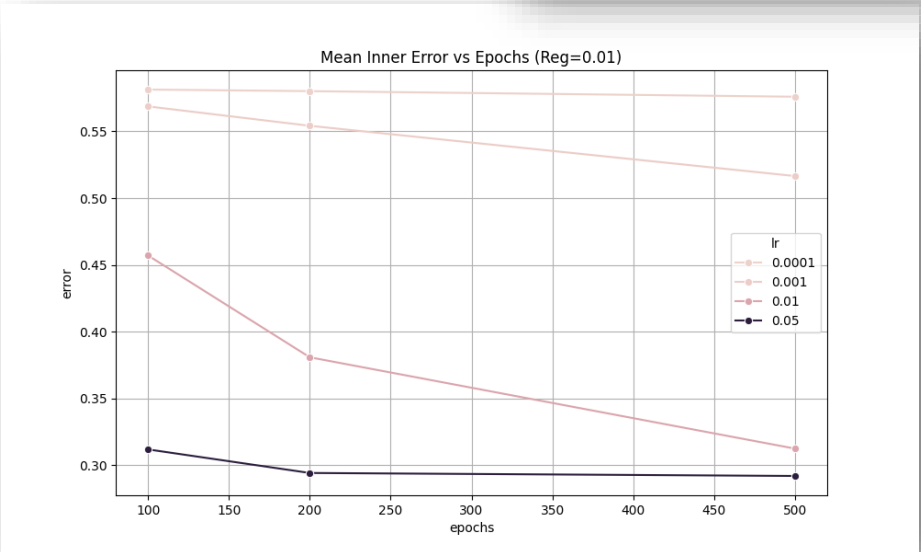
בגרפים הבאים ביצענו חישוב של דיוק המודל (למעשה – חישבנו את ה-mean inner error על פני חלוקות שונות ל-outer folds, עבור כל מודל M_i – שהוא היפותזה עם היפר-פרמטרים מסוימים של כל מודל).

רגרסיה לינארית

כיוון שברגרסיה ישנם 3 סוגי פרמטרים שונים, כדי לבדוק את התלות של השגיאה בכל אחד מהם – ציירנו 3 גרפים שונים מכל סוג.



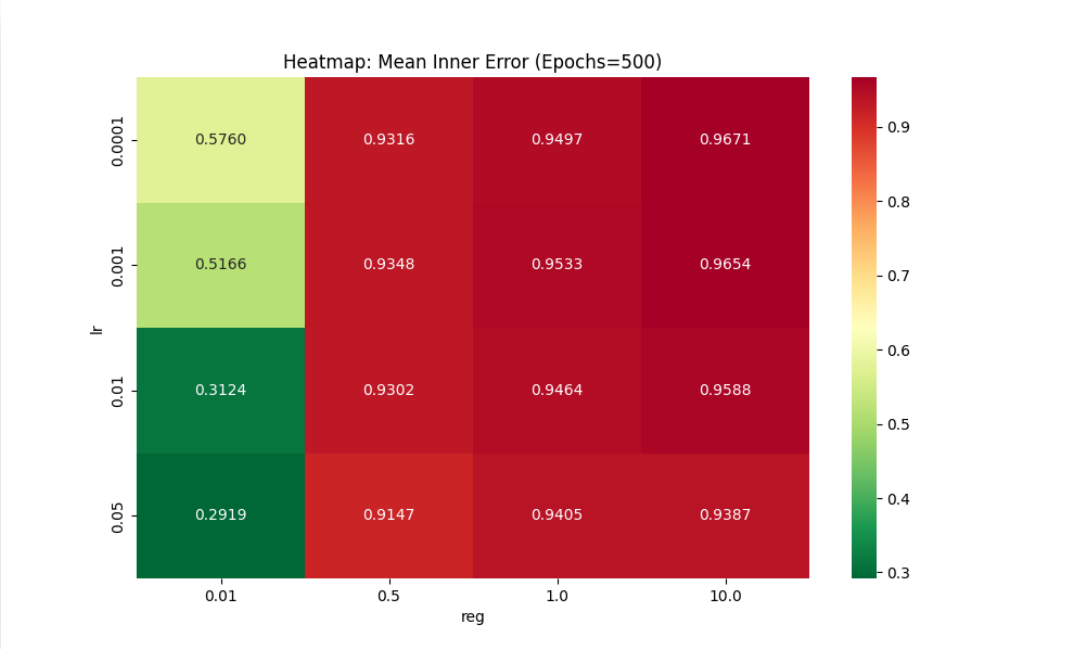


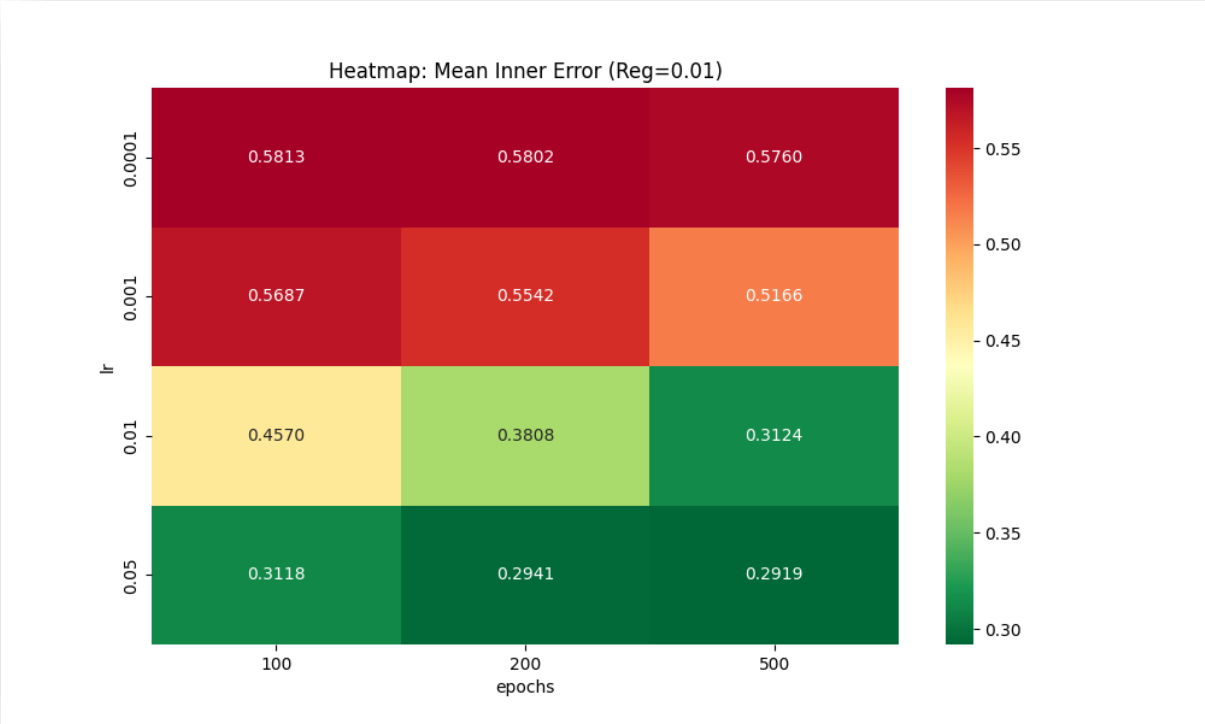


ניתן לראות שהכותרות מדברות בעד עצמן – בשני הגרפים העליונים קבענו את $Epochs$ לערכו המקסימלי (500) ושינינו פעם את lr ופעם את reg לקבלת ה-mean-inner-errors השונות (הפרמטר שלא היה המשתנה הבלתי תלוי הופיע ב-$legend$ על ערכיו השונים).

בנוסף, על מנת להציג את התוצאות בצורה טובה יותר השתמשנו במפות חום, שבהן יש שני צירים בלתי תלויים וציר אחד של צבע שמייצג את ה-mean-inner-error של כל שילוב ערכים של שני פרמטרים, כאשר הפרמטר השלישי מקובע (בגרף העליון מקובע $Ephocs = 500$, בגרף התחתון זהו $reg = 0.01$). כמובן שבשני הגרפים רואים שהתוצאה הטובה ביותר היא mean-inner-error של 0.2919 שמתקבל עבור המודל בעל הערך הגבוה ביותר לפרמטרים $Epochs, lr$ והערך הנמוך ביותר לפרמטר reg.

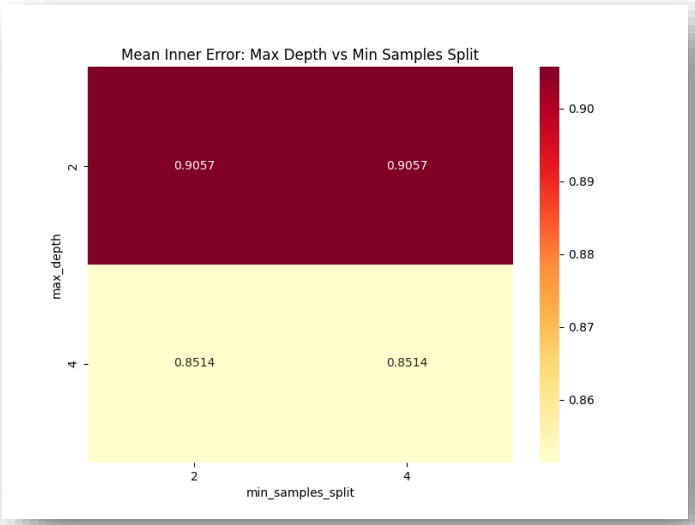
כדי להסביר את התוצאה, אנחנו משערים שהערך של lr צריך להיות מקסימלי כדי שבמספר ה-Epochs שבו השתמשנו המודל יצליח להגיע מספיק קרוב למינימום הלוקאלי שאליו הוא מכוון (כנראה שהיינו צריכים צעדים רבים כדי להגיע אליו במקרה הנוכחי) וכן שהערך של reg היה צריך להיות מינימלי פשוט כי לא בדקנו ערכים גבוהים מספיק שלו (רואים למשל במקרה של $lr = 0.05$ שישנה ירידה קלה מאוד בשגיאה בין $reg = 1$ ל-$reg = 10$). עם זאת, לא בדקנו את האופציות האלו בשל מגבלות של זמן ריצה וזמן עבודה על התרגיל, וניתן לראות שהצלחנו למרות הכל להגיע לתוצאה טובה יחסית עבור מודל לינארי פשוט – שגיאה של כ-29% בלבד בסיווג מורכב של וקטורים רבי כניסות ל-28 קבוצות שונות, ע"י מודל לינארי!

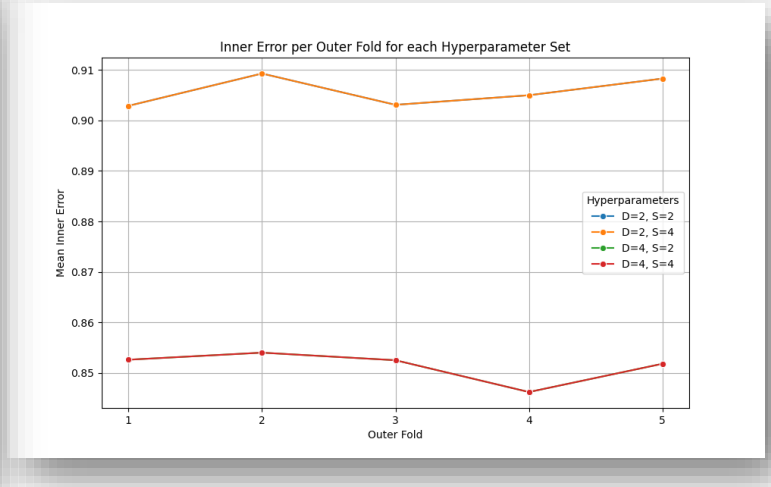




עץ

במקרה זה שינינו שני גדלים – את ה-min_samples_split ואת ה-max_depth. ניתן לראות שהפרמטר היחיד המשפיע על ה-mean-inner-error הינו ה-max_depth. לא התעמקנו בגרף זה או בניתוח שלו ע"י הקרוס-ואלידציה בשל פסילותו מראש כמודל רלוונטי עבור הפרויקט (לאחר ששמנו לב לזמני ריצה אדירים של האימון שלו גם עבור רמות נמוכות שכאלה, שהניבו לתוצאות לוקות מאוד שאף משתוות לביצועים של ה-unsupervised models מהחלק הבא).

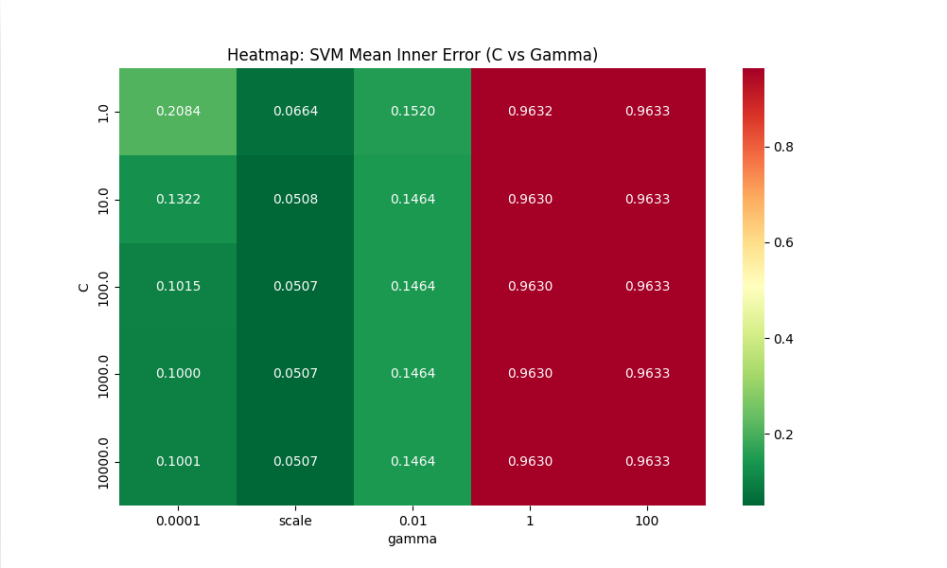


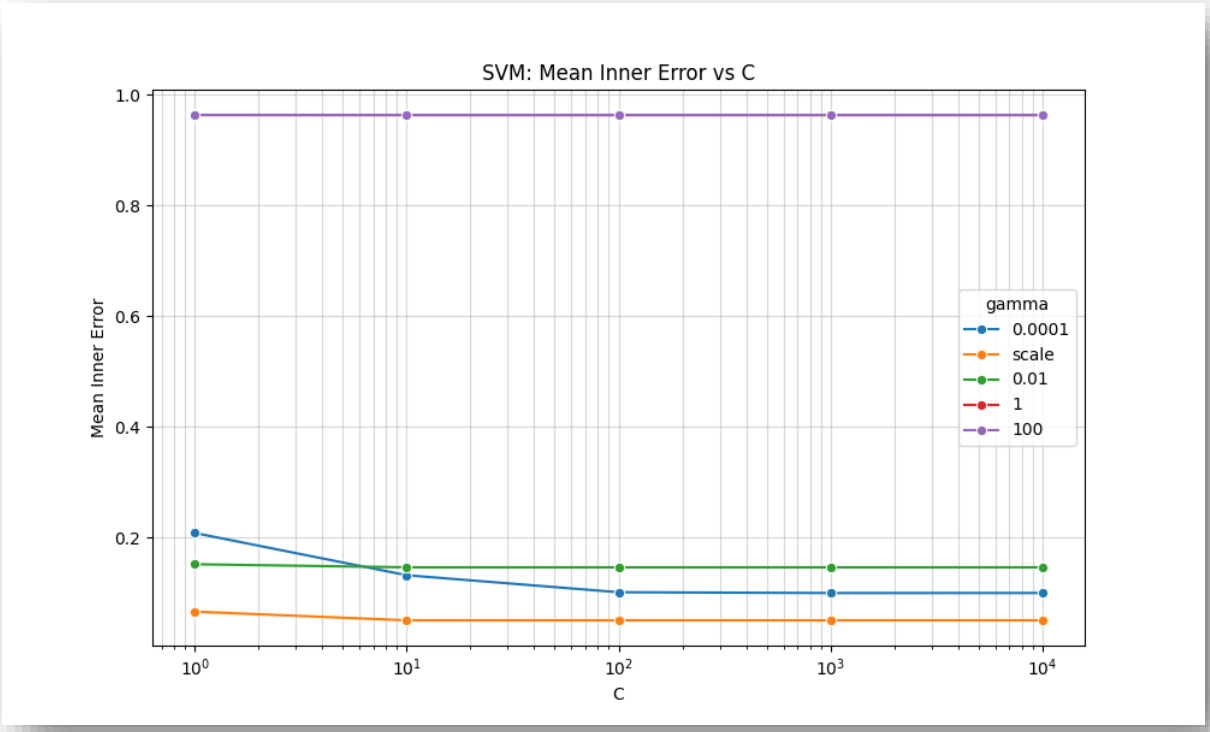


בגרף הימני, הגרף הכחול התלכד עם הגרף הכתום, והירוק עם האדום, כיוון שה-max_depth הוא הפרמטר היחיד שמשפיע על ה-mean-inner-error.

SVM

בגרף הבא ניתן לראות את התלות של ה-mean-inner-error במודלים השונים של ה-SVM עם זוגות הערכים השונים של (C, γ) שנבדקו. כפי שניתן לראות, התוצאות טובות במיוחד עבור $\gamma =' scale' \approx 1.1 \cdot 10^{-3}$, ואין שוני מבחינת mean-inner-error בין ערכי C שונים בגדולים מ-10.





להלן ההדפסה שהופיעה בסוף ריצת הקובץ partD_main.py שמופיע אצלנו בפרויקט. ניתן לראות עבור כל outer-fold את המודל הטוב ביותר שנמצא עבור אותו outer-fold ואת השגיאה שהתאימה לו. ניתן לראות שמדובר במודל בעל $C = 10$ שהגיע לתוצאות הטובות ביותר.

```
=== Nested CV: SVM (K_out=5, K_in=3) ===
Mean outer error: 0.0382 +/- 0.0034
  Outer fold 1: error=0.0409 | best params={'C': 10.0, 'gamma': 'scale'}
  Outer fold 2: error=0.0350 | best params={'C': 100.0, 'gamma': 'scale'}
  Outer fold 3: error=0.0378 | best params={'C': 10.0, 'gamma': 'scale'}
  Outer fold 4: error=0.0425 | best params={'C': 10.0, 'gamma': 'scale'}
  Outer fold 5: error=0.0350 | best params={'C': 10.0, 'gamma': 'scale'}
```

לכן, בחרנו במודל עבורו $C = 10, \gamma = 'scale'$ והשתמשנו בו על מנת לאמן עליו את כל הדאטה ואז לנבא באופן סופי את התגיות של קבוצת ה-Test, וקיבלנו את השורה הבאה:

```
Accuracy of the model over the training data: 0.999938
```

רגע של יושרה מקצועית – לא בדיוק הבנו מדוע דווקא $\gamma = \text{"scale"} \approx 1.1 \cdot 10^{-3}$ הוא הערך הטוב ביותר במודל, ולפי ה-gemini זה נובע מהמימוש של שיטת ה-SVM והגדרת ה-Kernel לפיה $K(x, x') = \exp(-\gamma \|x - x'\|^2)$ שמוביל לרגישות גבוהה לערך של γ, אשר במקרה שלנו הובילה לכך ש-$\gamma \approx 10^{-3}$ מניב תוצאות טובות הרבה יותר מאשר $\gamma \approx 10^{-4}$ או $\gamma \approx 10^{-2}$.

בנוסף, לא ברור לנו כיצד ייתכן שקיבלנו שגיאה טובה כל כך על קבוצת ה-Train עבור מודל זה (שגיאה קטנה ב-3 סדרי גודל מהשגיאות על ה-outer folds) – וזאת למרות שוידאנו את הנעשה בקוד. האם אימון על כלל דאטה האימון שיפר עד כדי כך את המודל? (כל outer fold מהווה רק 80% מהדאטה). בשל היותו של ה-SVM שייבאנו קופסה שחורה מבחינתנו, הנחנו שלא מדובר בתופעה חריגה במיוחד (גם בדקנו את השורה הנ"ל שהודפסה עבור מודלים גרועים יותר וקיבלנו תוצאות נמוכות יותר, מה שהוביל למסקנה שהוא כנראה נכון).

Part E – Attempting Unsupervised Methods

1. <u>המודל והצגת הדאטה</u>

בחרנו להשתמש במודל K-means שמבצע חלוקה של דאטה לא-מתויג ל-K קבוצות על בסיס ניחוש התחלתי של K מרכזים ב-R^d (d הוא כמות הפיצ׳רים), שיוך כל דוגמה למרכז הקרוב ביותר אליה ואז עדכון לפי המרכזים לפי ממוצעי הדוגמאות ששויכו לכל קבוצה.

הדאטה במקרה שלנו מבוסס על מוצא אלגוריתם ה-HOG עבור כל תמונה, שמגדיר לכל תמונה וקטור של כ-1000 פיצ׳רים, אשר באמצעות PCA נדחס ל-50 פיצ׳רים עבור כל תמונה.

על הוקטורים האלו אנחנו מבצעים את האלגוריתם, כאשר במקרה שלנו כמובן שנבחר ב-$K = 28$ שהוא מספר הקבוצות הטבעיות בבעיה (מספר התגיות, כמות סוגי האנשים).

על מנת לבדוק את דיוק המודל, ביצענו קישור בין התגית של כל דוגמה לקבוצה שהיא שויכה אליה ע״י ה-K-means באופן הבא: עבור כל קבוצה שמצא ה-K-means (שמוגדרת ע״י אחד מ-28 המרכזים שמצא האלגוריתם), שייכנו אותה לתגית השכיחה ביותר מבין כל התגיות שמתאימות לדוגמאות ששייכות לאותה הקבוצה. לאחר מכן, חישבנו את ממוצע הדיוק של כל 28 הקבוצות יחד – האחוז הממוצע של התגיות השכיחות ביותר שהתאימו לתגית שנמצאה בשיטה שתוארה לעיל.

2. <u>תוצר גולמי</u>

אחוז ההתאמה הממוצע של הקבוצות שמצא אלגוריתם ה-K-means לתגיות האמיתיות הוא 12.8% - כלומר 13.4% מהדגימות מקיימות את התנאי שאם אלגוריתם ה-K-means סיווג אותן לקבוצה מסוימת שעבורה נקבעה תגית מסוימת (עפ״י התגית השכיחה בקבוצה הזו), אז התגית שנקבעה זהה לתגית המקורית עבור אותה דגימה.

נשים לב שסיווג רנדומלי של כל דגימה לתגית מסוימת היה נותן אחוז דיוק של $\frac{1}{28} = 3.57\%$ (התפלגות אחידה על פני מרחב התגיות האפשריות). בנוסף, בדקנו זאת וקיבלנו שגם בשיטה של התאמת תגית רנדומלית לכל קבוצה שמצא ה-K-means (במקום לבחור בתגית השכיחה עבור אותה קבוצה) מקבלים דיוק של 9.9% - שהוא גם גדול משמעותית מאחוז הדיוק של הסיווג הרנדומלי.

נסיק שאלגוריתם K-means אכן מצליח לזהות קשרים בין הקבוצות השונות במרחב הדגימות (והוא עושה זאת באופן שטוב כמעט פי 4 מאשר הסיווג הרנדומלי, שהוא גרוע מאוד), אך הוא עדיין לא עושה זאת בצורה טובה מספיק.

3. <u>זיהוי מבנה פנימי של הדאטה</u>

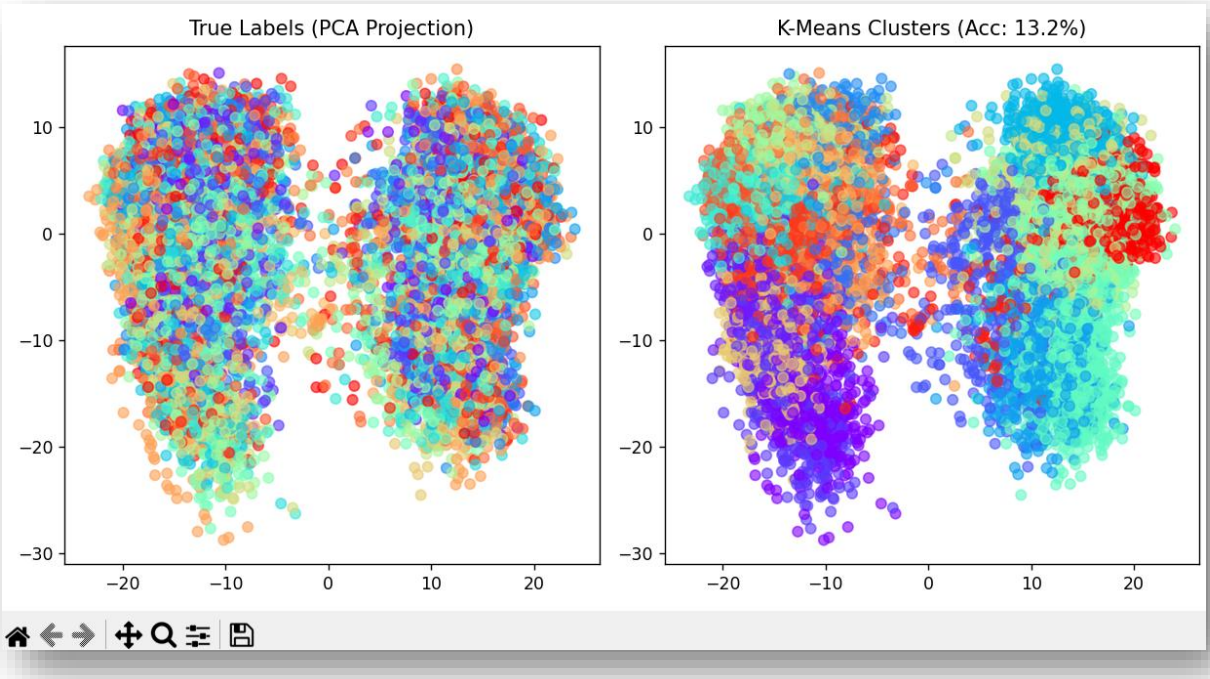
על מנת להציג את התוצאות, ביצענו אלגוריתם PCA על מטריצת הדגימות X לקבלת ייצוג בתת מרחב דו-מימדי עבור כל דגימה. לאחר מכן, הצגנו את כל הדגימות ב-2 גרפים. בכל אחד מהגרפים הצגנו את הדגימות בצבע עפ״י התגית שהותאמה להם – בגרף השמאלי ע״י התגיות האמיתיות, ובגרף הימני ע״י אלגוריתם K-means.

כפי שניתן להתרשם מהגרפים, בגרף הימני יש חלוקה גיאוגרפית של התגיות (פורמלית – הסיכוי של שתי דגימות להיות בעלות אותה התגית קורלטיבי למרחק ביניהן בתת המרחב הדו-מימדי הזה) כפי שהיינו מצפים מאלגוריתם K-means שעפ"י הגדרה מסווג תגיות על פי קירבה ביניהם ב-$\mathbb{R}^d$.

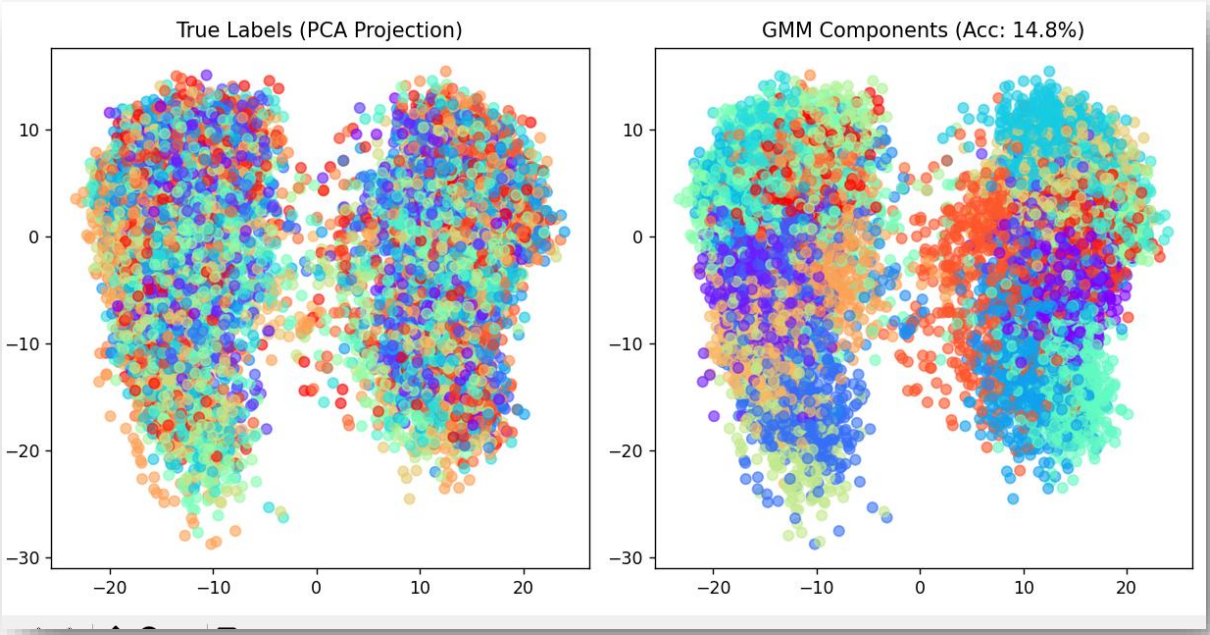
עם זאת, לא ניתן לטעון לקיומן של תבניות גיאומטריות ייחודיות.

4. <u>הערכתנו בנוגע למגבלות אלגוריתמים $unsupervised$:</u>

אנו מעריכים שהביצועים הגרועים של אלגוריתם ה-K-means עבור סיווג הדאטה שלנו נבעו מזה שמדובר באלגוריתם unsupervised אשר מנסה לסווג קבוצה גדולה מאוד (מעל 16,000) של וקטורים בעלי 50 כניסות. עפ"י מרחקים בלבד במרחב רב-מימדי. כיוון שאין שימוש בתגיות הנתונות (של דאטה האימון) לא היינו מצפי לאחוזי דיוק גבוהים כמו באלגוריתמים ה-supervised שכמובן עושים שימוש בדאטה כדי לשפר את ההיפר פרמטרים שלהם ואת הביצועים שלהם על פני קבוצות ה-validation וה-test.
עם זאת, במקרים רבים (ואפילו כאן) ניתן לקבל חלוקה הגיונית כלשהי לקבוצות על ידי ה-K-means, לפחות בצורה ראשונית ובזמן ריצה נמוך מאוד (פחות מ-10 שניות!).



לשם השוואה, השתמשנו בספריית GaussianMixture מתוך sklearn.mixture אשר מממשת את אלגוריתם GMM, והרצנו גם עליה את כל הדאטה (מתויג ולא מתויג גם יחד) כדי להשתמש בה לביצוע סיווג של הדאטה עפ"י תגית אמת שכיחה ביותר, והגענו לאחוזי דיוק דומים לאלגוריתם ה-K-means, של 14.8% כפי שנראה בגרף שלהלן:



כאמור השימוש באלגוריתם unsupervised עבור דאטהסט עצום כל כך, ובהתחשב בקורלציה שהיא אינטואיטיבית די נמוכה בין המרחק של שני וקטורי HOG ב-$\mathbb{R}^{50}$ לבין הסיכוי ששתי דגימות נמצאות באותה הקבוצה (קורלציה שעליהן מתבססים שני האלגוריתמים שנידונו כאן) – סביר שלא הצלחנו להגיע לאחוזי דיוק מרשימים או ללמוד תבניות מיוחדות על הדאטה שלא נובעות מהעדיפות של המודל לסווג יחד גושים של נקודות קרובות על פני תת המרחב הדו-מימדי (תופעה שניתן לשים אליה לב בקלות בשני תתי-הגרפים הימניים בכל גרף).

דגש: את אלגוריתם ה-K-means מימשנו בתרגיל (באמצעות כלי AI) ובשביל אלגוריתם GMM השתמשנו בספרייה מובנית.

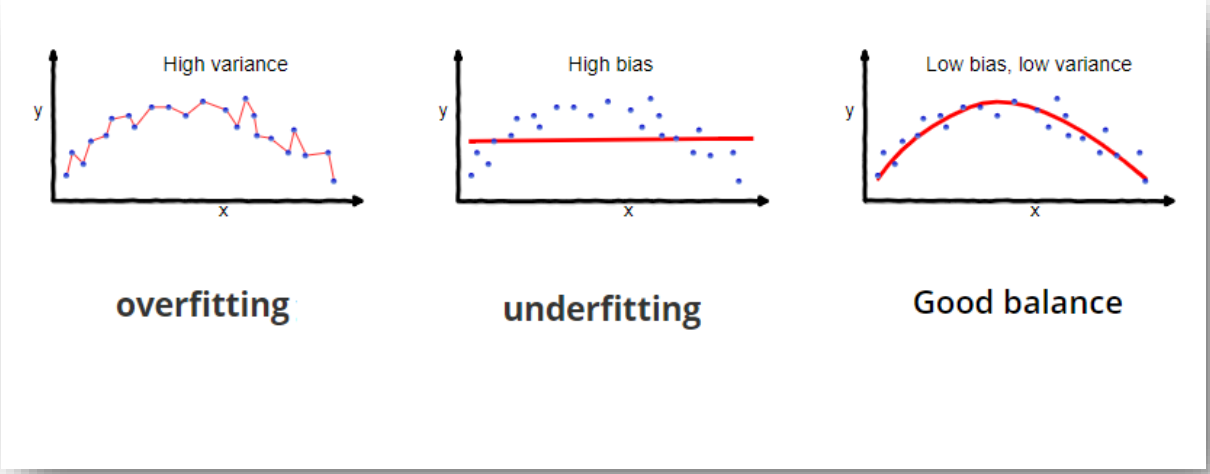
Part F – Theoretical Discussion: Bias-Variance Tradeoff

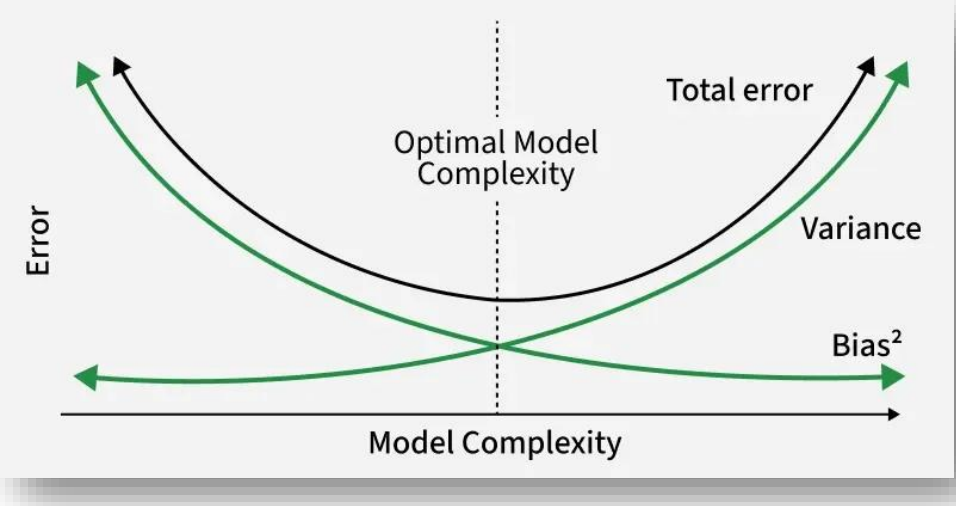
<u>בקצרה</u>:

Lots of assumptions -> high bias -> underfitting

High sensitivity -> high variance -> overfitting

ביאס, היסט, מודד כמה בעצם אנחנו מגיעים להיות סביב ההיפותזה הנכונה. וריאנס, שונות, מודד בעצם כמה המודל רגיש, כלומר כמה הפיזור עבור סטים שונים של נתונים הוא גבוה. יש tradeoff בינהם- ככל שהמודל נהיה מאוד טוב עבור סט נתונים מסוים, הוא יהיה יותר מדי ספציפי, כמו בתמונה השמאלית. וככל שהוא כללי מדי ורלוונטי לכמה סטים, הוא יכול לא לקבל את הצורה הנכונה בכלל, כמו בתמונה האמצעית.





בפרוייקט הספציפי הזה:

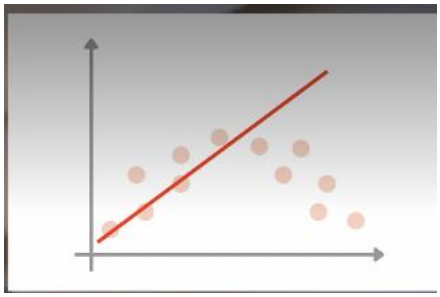
1. מבנה

יש לנו 28 אנשים (מחלקות). ככל שיש יותר אנשים, הבעיה דורשת מודל יותר גמיש וכללי כדי להבדיל בין תווי פנים דומים. זה מעלה את הסיכון ל-variance גבוה, כי המודל עלול להינעל על פיקסל מקרי שמבדיל בין שני אנשים דומים בסט האימון בלבד, ובעצם לא להצליח להבדיל בין האנשים בהמשך.

הפרעות סינתטיות:

התמונות כללו רעש, שינויי תאורה או טשטוש לתמונות, וזה מעלה את ה-**Variance** של המודל, כי הוא עלול לנסות ללמוד את הרעש הסינתטי כתכונה מייצגת של האדם. כדי להתגבר על זה, אנחנו חייבים **רגולריזציה חזקה יותר** שתכריח את המודל להתעלם מהפרטים הקטנים (הרעש) ולהתמקד במבנה הכללי של הפנים. ב/שביל זה עיבדנו את הנתונים באופן מסוים, וגם נעשה רגולציה נכונה על הפרמטרים.

במודל הליניארי:



מכיוון שהוא מניח באופן אוטומטי שההיפותזה תהיה ליניארית, הוא יכול לעשות undefitting חמור, ולא להצליח להתאים את המודל באופן מוצלח.

לעומת זאת, השונות שלו נמוכה, מכיוון שהוא לא משתנה דרמטית בעקבות שינוי בנתונים, ושומר על צורה יחסית קבועה.

במודל של העץ:

היפר־פרמטרים שמגבילים את גודל העץ מגדילים את הביאס ומקטינים את הוריאנס ,בעוד שהיפר־פרמטרים המתירים לעץ לגדול ללא הגבלה מקטינים את הביאס אך מגדילים את הוריאנס.

באופן טבעי, יהיה לו bias נמוך, מכיוון שהוא יכול להתאים את עצמו טוב מאוד - אם הוא היה יכול לגדול ללא הגבלה, העץ היה יכול להגיע לכל תשובה אפשרית במדויק. זה היה יוצר overfitting, ולכן מגבילים את גודל העץ. לעומת זאת, יש לו variance גבוה.

במודל ה-SVM, יש שני פרמטרים, שמשפיעים ישירות על הביאס והשונות של המודל.

- C נמוך = רגולריזציה חזקה:
  - המודל מחפש מרווח (Margin) רחב ככל האפשר, ומוכן להתעלם מנקודות בודדות שחורגות.
  - לכן, המודל יוצא פשטני, וה-variance נמוך למרות שהביאס יותר גבוה.
- C גבוה = רגולריזציה חלשה:
  - המודל מנסה לסווג כל נקודה בצורה מושלמת.

- המרווח נהיה צר מאוד ומפותל כדי "לעקוף" נקודות קשות.
- התוצאה Bias: נמוך (המודל מדויק על האימון) ו-**Variance** גבוה כי המודל רגיש.

- גאמא נמוך = רגולציה חזקה:
  - כל תמונה משפיעה על אזור רחב מאוד בתוך המרחב.
  - גבול ההחלטה יהיה חלק, ליניארי
  - התוצאה Bias: גבוה ו Variance-נמוך.
- גאמא גבוה = רגולציה חלשה:
  - כל תמונה משפיעה רק על הסביבה המיידית שלה.
  - המודל יוצר "בועות" קטנות מסביב לכל דגימה.
  - ביאס נמוך, וריאנס גבוה.

Final Conclusion

במהלך התרגיל ביצענו את ה-pipeline המלא בפתרון של בעיית קלאסיפיקציה של פרצופים לאנשים ע״י מודל למידת מכונה קלאסי.

בחלק A דנו בשיטה שבחרנו לביצוע ה-preprocessing שהתבסס על 3 שלבים עיקריים:

1. שימוש באלגוריתם CLAHE להגברת הניגודיות בתמונה כהכנה ללמידה.
2. שימוש באלגוריתם HOG להוצאת פיצ׳ירם על התמונה שאמורים להיות בעלי משמעות הקשורה לצורה של התמונה ותבניות מיוחדות שמופיעות בפרצוף.
3. שימוש באלגוריתם PCA כדי לכווץ את הדאטה שעליו מאומנים המודלים המוצעים.

בחלק B הצגנו את 3 מודלי ה-ML שבהם השתמשנו בתרגיל, ודנו בנושאים הקשורים אליהם כגון היפר-פרמטרים רלוונטיים, ההשפעה שלהם ושל רגולריזציה על ה-bias\variance של המודלים ועל הרלוונטיות שלהם למשימת הסיווג שנדרשנו לעמוד בה בתרגיל.

בחלק C ביצענו מימוש בפועל של שניים מתוך 3 המודלים שהצענו (ושימוש בספרייה מוכרת עבור השלישי, ה-SVM עם ה-kernel). בחלק זה דאגנו גם לשנות את ההיפרפרמטרים עבור כל משפחת היפותזות ולהסביר כיצד ה-plots עבור הדיוק של המודלים כתלות בערכם של אותם פרמטרים מאושש את ההסברים שלנו בנוגע להשפעה של הפרמטרים על ה-bias\variance של המודלים.

לאחר מכן, בחלק D ביצענו את ההשוואה הזו בצורה סיסטמטית – ע״י אימון נפרד של המודלים בצורת Cross-Validation, עם הטייה מינימלית (ע״י שימוש ב-outer folds) וניסוי היפר-פרמטרים בסקאלה גבוה יחסית עבור כל משפחת היפותזות בנפרד, ומציאה ממנה של ההיפותזה (מודל + פרמטרים) הטובה ביותר.

בחלק E ניסינו להשתמש עם שני מודלים unsupervised ולברר איזו חלוקה הם מסוגלים לבצע על הדאטה, ומה אחוזי הדיוק של החלוקה הזו עם החלוקה הטבעית של הדאטה (באמצעות זהות האדם שפרצופו מופיע בתמונה) – הגענו לדיוק מסדר גודל של 10%, ודנו בסיבות לכך ובמשמעויות.

לבסוף, בחלק F כתבנו הסבר תיאורטי נרחב יותר של ההתייחסות ל-bias\variance tradeoff באופן כללי במודלים של למידת מכונה, ובפרט בפרויקט זה.

אם נסכם את התהליך שעברנו – למדנו כאן איך לממש בכוחות עצמנו pipeline שלם של פתרון בעייה סטנדרטית נפוצה ושימושית מאוד במדעי הנתונים. חקרנו על מודלים מורכבים (גם כאלה שלא הופיעו בקורס או שמימושם לא הופיע כמו ה-SVM), על אופן הפעולה ועל ההיפר-פרמטרים שלהם, ולאחר מכן ביצענו מבחן השוואה ריגורוזי בין המודלים השונים – ולמעשה נתנו למחשב לבחור את אלו שקיבלו את הציון (mean outer error) הטוב ביותר.

ואם כבר באבליואציה עסקינן – אז התוצאה הסופית הייתה התאמה של 96.4%! במודל המתאים לתוצאה זו השתמשנו על לאמן אותו על הדאטה כולו ולחזות באמצעותו את הפרצופים בדאטה של ה- Test (כמובן שלא נגענו בדאטה זה לכל אורך התרגיל, כדי לקבל תוצאות כמה שפחות מוטות ולהצליח בבעיית ההכללה).

את התוצאות שמרנו בקובץ נפרד כפי שמתבקש בהוראות התרגיל.